Argustack Estimate Tool — Deep Dive

Version 0.1.12 | March 2026

How Argustack predicts task duration: algorithm, data sources, scoring, and output interpretation.

Visit the project on GitHub or install from npm.

> **Note:** This documentation is actively maintained alongside the codebase. While we verify accuracy with each release, minor discrepancies may exist as features evolve. For the latest information, refer to the GitHub repository. Found an issue? Open a ticket.

Table of Contents

1. What It Does
2. Two Operating Modes
3. Data Source Priority
4. Finding Similar Tasks
5. Calculating Base Hours
6. Developer Coefficients
7. Familiarity Factor
8. Bug Overhead
9. Final Prediction
10. CSV-Only Mode
11. Improving Accuracy

1. What It Does

The estimate tool answers one question: **"How long will it take this developer to complete this task?"**

It works by analyzing your project's actual history — not industry averages, not story points, not gut feeling. Real data from your team, your codebase, your workflow.

Input: Task description + developer name + optional filters (type, components)

Output:

- Similar completed tasks with actual effort
- Developer-specific speed coefficient
- Component familiarity factor
- Two predictions: **without bugs** and **with bugs**

2. Two Operating Modes

The tool automatically detects what data is available and adjusts its output:

Mode	Data Available	Output
Full Prediction	Jira API + Git (time tracking, worklogs, commit spans)	Hours per developer, two predictions
Resolution Timeline	CSV only (no effort data)	Business days range, lead time

Why two modes?

Effort data (time_spent, worklogs, commit history) tells you how many hours someone actually worked on a task. This enables per-developer predictions.

Cycle time (created → resolved) tells you how many calendar days a task took from creation to closure. This includes backlog wait time, code review time, and any other delays — not just active development.

Multiplying a developer's speed coefficient by cycle time would be meaningless (you can't speed up backlog wait time), so the tool switches to a different output format when only cycle time is available.

3. Data Source Priority

For each similar task, the tool looks for effort data in this order:

Priority	Source	What it measures	Where it comes from
1	`time_spent`	Actual hours logged	Jira issue field
2	Worklogs	Detailed time entries	Jira worklog API
3	Commit span	First → last commit	Git commit timestamps
4	`original_estimate`	Manager's estimate	Jira issue field
5	Cycle time	Created → resolved	Jira dates (business days)

The tool uses the highest-priority source available. If a task has `time_spent` logged, it uses that. If not, it checks worklogs. If no worklogs, it looks at commit history. And so on.

When **all** similar tasks fall back to cycle time (priority 5), the tool switches to Resolution Timeline mode.

Business Days Calculation

Cycle time is converted to business days — weekdays only, 8 hours per day, weekends excluded. This prevents inflated estimates from tasks that sat in the backlog over weekends or holidays.

```
Monday → Friday = 5 business days (40h)
Monday → Monday (next week) = 6 business days (48h), not 8 calendar days
```

4. Finding Similar Tasks

The tool searches for completed tasks similar to the one being estimated using a **composite scoring** algorithm.

Scoring Weights

Signal	Weight	How it works
Text relevance	30%	PostgreSQL full-text search (`ts_rank`) on task descriptions
Type match	25%	Bug → Bug, Task → Task (binary: 1.0 or 0.0)
Component overlap	35%	Fraction of matching components
Temporal recency	10%	Recent tasks weighted higher: `1 / (1 + years_ago)`

How weights adapt

When optional parameters aren't provided, their weight becomes 0 and the remaining signals are used as-is:

Parameters provided	Effective scoring
description only	Text 30%, recency 10%
description + issue_type	Text 30%, type 25%, recency 10%
description + components	Text 30%, components 35%, recency 10%
all parameters	Text 30%, type 25%, components 35%, recency 10%

Filters

Only completed tasks are considered as analogs:

- `status_category = 'Done'` — must be resolved
- `status != 'Canceled'` — not canceled tickets
- Optionally excludes a specific issue key (useful when estimating a task that already exists in the database)

Example

Estimating: "Fix payment export duplicates" (Bug, components: [Export, Payments])

```
Score breakdown for PROJ-100 "Fix CSV export crash":
  Text match:     0.72 × 0.30 = 0.216
  Type match:     1.00 × 0.25 = 0.250  (both Bugs)
  Components:     0.50 × 0.35 = 0.175  (1 of 2 components match)
  Recency:        0.85 × 0.10 = 0.085  (resolved 2 months ago)
  ──────────────────────────────────────
  Composite score:              0.726
```

5. Calculating Base Hours

Once similar tasks are found, their effort hours are combined into a single base estimate.

Weighted Trimmed Mean

1. **Sort** all task hours ascending
2. **Trim** outliers — remove top and bottom 10% (for 6+ tasks)
3. **Weight** by temporal recency — recent tasks matter more
4. **Calculate** weighted average

Step	Tasks: 2h, 3h, 4h, 4h, 5h, 6h, 8h, 15h
Sort	2, 3, 4, 4, 5, 6, 8, 15
Trim 10%	3, 4, 4, 5, 6, 8 (removed 2h and 15h)
Weight	Recent tasks get higher weight
Result	~4.8h (weighted trimmed mean)

Why trimmed mean?

Raw averages are distorted by outliers. A task that took 15h due to scope creep shouldn't pull up the prediction for a normal 4h task. Trimming the extremes gives a more robust central estimate.

Fallback

If there are 5 or fewer similar tasks, no trimming is applied — the dataset is too small. If temporal weights are unavailable, a simple average is used.

6. Developer Coefficients

Each developer has a personal speed multiplier based on their full history of completed tasks.

How it's calculated

For each developer with 3+ completed tasks:

```
coefficient = MEDIAN(actual_hours / estimated_hours)
```

Using median (not average) because it's robust against outliers.

Two coefficients

Coefficient	Formula	What it means
Without bugs	`MEDIAN(time_spent / original_estimate)`	Pure development speed
With bugs	`MEDIAN((time_spent + bug_time) / original_estimate)`	Real cost including bug fixes

Context-specific cascade

The tool tries to find the most relevant coefficient:

Priority	Context	Example
1	Developer + issue type	"Sarah's Bug coefficient"
2	Developer + all types	"Sarah's global coefficient"

If the developer has enough data (3+ tasks) for a type-specific coefficient, that's used. Otherwise, it falls back to their global coefficient across all types.

Outlier exclusion

Tasks where `time_spent / original_estimate > 5.0` are excluded from coefficient calculation. These represent scope creep (a 4h estimate that turned into 20h of work) and would unfairly skew the developer's coefficient.

Example

```
Sarah's Bug coefficient:
  Task 1: 4h actual / 4h estimate = 1.00
  Task 2: 3h actual / 4h estimate = 0.75
  Task 3: 5h actual / 4h estimate = 1.25
  Task 4: 3h actual / 2h estimate = 1.50
  Task 5: 2h actual / 4h estimate = 0.50

  Median = 1.00 → Sarah completes Bug estimates in roughly the estimated time

  With bugs:
  Task 1: (4h + 1h bug) / 4h = 1.25
  Task 2: (3h + 0h bug) / 4h = 0.75
  Task 3: (5h + 2h bug) / 4h = 1.75
  Task 4: (3h + 0h bug) / 2h = 1.50
  Task 5: (2h + 1h bug) / 4h = 0.75

  Median = 1.25 → Bugs add ~25% overhead to Sarah's work
```

7. Familiarity Factor

Developers who have worked on similar components before are faster. The familiarity factor quantifies this.

Formula

```
factor = max(0.6, 1.0 − 0.08 × resolved_count)
```

Resolved tasks in component	Factor	Meaning
0 (first time)	1.00	No discount
1	0.92	8% faster
3	0.76	24% faster
5+	0.60	40% faster (max)

How it works

1. Query: "How many tasks has this developer resolved in each component?"
2. Match: Which components overlap with the task being estimated?
3. Sum: Total resolved tasks across matching components
4. Apply formula: More experience → lower factor → faster estimate

Example

Estimating a task in components [Export, Payments] for Sarah:

```
Sarah's component history:
  Export:   5 resolved tasks
  Payments: 2 resolved tasks
  Auth:     8 resolved tasks (not relevant)

Matching components: Export(5) + Payments(2) = 7 total
Factor: max(0.6, 1.0 − 0.08 × 7) = max(0.6, 0.44) = 0.60

Sarah is very familiar with this area — 40% discount applied
```

When it's not applied

- No components specified in the estimate request → factor = 1.0
- Developer has no history in any matching component → factor = 1.0
- No component data in the database → factor = 1.0

8. Bug Overhead

Every task has a hidden cost: bugs that come after it. The estimate tool quantifies this separately.

How bugs are linked

Two sources of bug associations:

Link type	How it works
Parent-child	Bug is a sub-task of the story
Issue links	Bug is linked to the story (blocks, caused by, etc.)

Bug time calculation

For each similar task:

1. Find all child/linked issues of type "Bug" or "Sub-bug"
2. Sum their `time_spent` values
3. Calculate: `real_cost = task_time + bug_time`
4. Bug coefficient: `(task_time + bug_time) / original_estimate`

What it reveals

```
Base estimate: 8h
Coefficient without bugs: ×0.90 → 7.2h
Coefficient with bugs:    ×1.15 → 9.2h

Bug overhead: 28% ((9.2 − 7.2) / 7.2)

This developer's features typically generate bugs that cost
28% of the original development time to fix.
```

9. Final Prediction

All factors combine into the final prediction:

Formula

```
prediction = base_hours × developer_coefficient × familiarity_factor
```

Two predictions

Prediction	Formula	Use case
Without bugs	base × coeff_no_bugs × familiarity	Optimistic: pure development time
With bugs	base × coeff_with_bugs × familiarity	Realistic: includes bug aftermath

Full example

```
Task: "Fix payment export duplicates"
Developer: Sarah
Type: Bug
Components: [Export, Payments]

Step 1 — Similar tasks found: 8
Step 2 — Base hours: 4.2h (weighted trimmed mean, 7/8 tasks after trim)
Step 3 — Sarah's Bug coefficient: ×0.85 (no bugs), ×1.05 (with bugs)
Step 4 — Familiarity: ×0.72 (9 resolved in Export + Payments)
Step 5 — Combine:

Without bugs: 4.2h × 0.85 × 0.72 = 2.6h (0.3 days)
With bugs:    4.2h × 1.05 × 0.72 = 3.2h (0.4 days)
Bug overhead: +23%
```

When data is incomplete

Missing data	Behavior
No developer coefficient	Shows base hours without multiplier
No familiarity data	Factor = 1.0 (no discount)
No similar tasks	"No similar completed tasks found"
Fewer than 3 tasks for coefficient	"Need 3+ completed tasks" + raw base hours

10. CSV-Only Mode

When you import issues from a Jira CSV export (instead of using the Jira API), there's no effort tracking data — no `time_spent`, no worklogs, no original estimates. The only time signal is cycle time (created → resolved).

What changes

Aspect	Full Mode	CSV-Only Mode
Time source	Actual work hours	Calendar days (created → resolved)
Developer coefficient	Applied	**Not applied**
Familiarity factor	Applied	**Not applied**
Output format	Hours per developer	Business days range
Label	"Prediction"	"Resolution Timeline"

Why no developer coefficients?

Cycle time includes backlog wait time. A task might sit in "To Do" for 3 weeks before anyone touches it. Multiplying that by a developer's speed coefficient would be meaningless — you can't speed up waiting.

Output format

```
## Resolution Timeline (cycle time only)

Similar tasks were closed in 5–8 business days from creation.

This is lead time (backlog wait + development + code review),
NOT active development time.

⚠ No effort tracking data available (time_spent, worklogs, commit history).
Connect Jira API or Git for actual work hours and per-developer predictions.
```

Business days calculation

Only weekdays count. 8 hours per business day.

```
Task created: Monday Jan 6
Task resolved: Friday Jan 17

Calendar days: 12
Business days: 10 (excludes Saturday Jan 11 and Sunday Jan 12)
Business hours: 80h
```

11. Improving Accuracy

The estimate tool gets better with more data. Here's how to maximize accuracy:

Connect more data sources

Source	What it adds	Impact on accuracy
Jira API (instead of CSV)	time_spent, worklogs, estimates	Enables per-developer predictions
Git	Commit timestamps, file changes	Adds coding span as effort signal
GitHub	PR timelines, review data	Better cycle time breakdown

Use optional parameters

Parameter	What it improves
`issue_type`	Finds same-type analogs (Bug→Bug matching)
`components`	Finds tasks in the same code area

Team practices that help

Practice	Why it helps
Log time in Jira	Highest-priority effort signal
Include issue keys in commits	Enables commit span calculation
Include issue keys in PR titles	Links PRs to issues automatically
Use components in Jira	Enables familiarity factor
Set original estimates	Enables coefficient calculation

Data requirements

Feature	Minimum data needed
Basic prediction	1+ similar completed tasks with effort data
Developer coefficient	3+ completed tasks by the developer (with estimate + time_spent)
Familiarity factor	Developer history in matching components
Bug overhead	Bug issues linked as children or via issue links

Visit the project on **GitHub** or install from **npm**.